Your code will likely look something like this:

```
class MyParser[+A](...) { ... }

object MyParsers extends Parsers[MyParser] {
  // implementations of primitives go here
}
```

Replace `MyParser` with whatever data type you use for representing your parsers. When you have something you're satisfied with, get stuck, or want some more ideas, keep reading.

9.6.1 One possible implementation

We're now going to discuss an implementation of `Parsers`. Our parsing algebra supports a lot of features. Rather than jumping right to the final representation of `Parser`, we'll build it up gradually by inspecting the primitives of the algebra and reasoning about the information that will be required to support each one.

Let's begin with the `string` combinator:

```
def string(s: String): Parser[A]
```

We know we need to support the function `run`:

```
def run[A](p: Parser[A])(input: String): Either[ParseError,A]
```

As a first guess, we can assume that our `Parser` *is* simply the implementation of the `run` function:

```
type Parser[+A] = String => Either[ParseError,A]
```

We could use this to implement the `string` primitive:

```
def string(s: String): Parser[A] =
  (input: String) =>
    if (input.startsWith(s))
      Right(s)
    else
      Left(Location(input).toError("Expected: " + s))
```

Uses `toError`, defined later, to construct a `ParseError`

The `else` branch has to build up a `ParseError`. These are a little inconvenient to construct right now, so we've introduced a helper function, `toError`, on `Location`:

```
def toError(msg: String): ParseError =
  ParseError(List((this, msg)))
```

9.6.2 Sequencing parsers

So far, so good. We have a representation for `Parser` that at least supports `string`. Let's move on to sequencing of parsers. Unfortunately, to represent a parser like `"abra" ** "cadabra"`, our existing representation isn't going to suffice. If the parse of `"abra"` is successful, then we want to consider those characters *consumed* and run the